

Introduction to PyTorch - I

PyTorch is an open-source machine learning framework based on the Python programming language, used for building and training deep learning models. Widely adopted across academia and industry, PyTorch has become the framework of choice for cutting-edge research and commercial AI applications.

Key Features:

- **Tensors:** PyTorch uses tensors as its fundamental data structure. Tensors are multi-dimensional arrays, like [NumPy](#) arrays, but with the crucial ability to run on GPUs to accelerate computation.
- **Automatic Differentiation (Autograd):** The built-in autograd engine automatically calculates the gradients needed for training neural networks through backpropagation, which simplifies the process of optimizing model parameters.
- **Robust Ecosystem:** PyTorch has a rich ecosystem of libraries and tools that extend its functionality for specific domains, including:
 - TorchVision for computer vision tasks.
 - TorchText for natural language processing.
 - TorchServe for deploying models at scale.

Tensors

Tensors are a specialized data structure that are very similar to arrays and matrices. In PyTorch, we use tensors to encode the inputs and outputs of a model, as well as the model's parameters. Tensors are similar to [NumPy's](#) ndarrays, except that tensors can run on GPUs or other hardware accelerators.

```
import torch
import numpy as np
```

Initializing a Tensor:

Tensors can be initialized in multiple ways:

- a. **Directly from data:**
Tensors can be created directly from data. The data type is automatically inferred.

```
data = [[1, 2], [3, 4]]
x_data = torch.tensor(data)
```

b. From a Numpy Array:

```
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
```

c. From another tensor:

```
x_ones = torch.ones_like(x_data) # retains the properties of x_data
print(f"Ones Tensor: \n {x_ones} \n")

x_rand = torch.rand_like(x_data, dtype=torch.float) # overrides the datatype of x_data
print(f"Random Tensor: \n {x_rand} \n")
```

Out:

```
Ones Tensor:
tensor([[1, 1],
        [1, 1]])

Random Tensor:
tensor([[0.0153, 0.5753],
        [0.1464, 0.8541]])
```

d. With Random or constant values:

```
shape = (2,3,)
rand_tensor = torch.rand(shape)
ones_tensor = torch.ones(shape)
zeros_tensor = torch.zeros(shape)

print(f"Random Tensor: \n {rand_tensor} \n")
print(f"Ones Tensor: \n {ones_tensor} \n")
print(f"Zeros Tensor: \n {zeros_tensor}")
```

Out:

```
Random Tensor:
tensor([[0.3107, 0.1623, 0.1237],
        [0.7441, 0.1596, 0.2919]])

Ones Tensor:
tensor([[1., 1., 1.],
        [1., 1., 1.]])

Zeros Tensor:
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

e. Attributes of a Tensor:

Tensor attributes describe their shape, datatype, and the device on which they are stored.

```

tensor = torch.rand(3,4)

print(f"Shape of tensor: {tensor.shape}")
print(f"Datatype of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")

```

Out:

```

Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu

```

Operations on Tensor:

- a. Standard Numpy like indexing & Slicing:

```

tensor = torch.ones(4, 4)
print(f"First row: {tensor[0]}")
print(f"First column: {tensor[:, 0]}")
print(f"Last column: {tensor[:, -1]}")
tensor[:,1] = 0
print(tensor)

```

Out:

```

First row: tensor([1., 1., 1., 1.])
First column: tensor([1., 1., 1., 1.])
Last column: tensor([1., 1., 1., 1.])
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])

```

- b. Joining Tensors:

```

t1 = torch.cat([tensor, tensor, tensor], dim=1)
print(t1)

```

Out:

```

tensor([[1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
        [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
        [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
        [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.]])

```

c. Arithmetic Operations:

```
# This computes the matrix multiplication between two tensors. y1, y2, y3 will have the same value
# ``tensor.T`` returns the transpose of a tensor
y1 = tensor @ tensor.T
y2 = tensor.matmul(tensor.T)

y3 = torch.rand_like(y1)
torch.matmul(tensor, tensor.T, out=y3)

# This computes the element-wise product. z1, z2, z3 will have the same value
z1 = tensor * tensor
z2 = tensor.mul(tensor)

z3 = torch.rand_like(tensor)
torch.mul(tensor, tensor, out=z3)
```

Out:

```
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])
```

d. In-place operation:

```
print(f"{tensor} \n")
tensor.add_(5)
print(tensor)
```

Out:

```
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])

tensor([[6., 5., 6., 6.],
        [6., 5., 6., 6.],
        [6., 5., 6., 6.],
        [6., 5., 6., 6.]])
```

Bridge with Numpy:

Tensors on the CPU and NumPy arrays can share their underlying memory locations, and changing one will change the other.

a. Tensor to NumPy array:

```
t = torch.ones(5)
print(f"t: {t}")
n = t.numpy()
print(f"n: {n}")
```

Out:

```
t: tensor([1., 1., 1., 1., 1.])
n: [1. 1. 1. 1. 1.]
```

b. change in the tensor reflects in the NumPy array.

```
t.add_(1)
print(f"t: {t}")
print(f"n: {n}")
```

Out:

```
t: tensor([2., 2., 2., 2., 2.])
n: [2. 2. 2. 2. 2.]
```

c. Numpy Array to Tensor:

```
np.add(n, 1, out=n)
print(f"t: {t}")
print(f"n: {n}")
```

Out:

```
t: tensor([2., 2., 2., 2., 2.], dtype=torch.float64)
n: [2. 2. 2. 2. 2.]
```

Datasets & DataLoaders

Code for processing data samples can get messy and hard to maintain; we ideally want our dataset code to be decoupled from our model training code for better readability and modularity.

PyTorch provides two data primitives: **`torch.utils.data.DataLoader`** and **`torch.utils.data.Dataset`** that allows you to use pre-loaded datasets as well as your own data. Dataset stores the samples and their corresponding labels, and DataLoader wraps an iterable around the Dataset to enable easy access to the samples.

Loading a Dataset:

Here is an example of how to load the [Fashion-MNIST](#) dataset from TorchVision. Fashion-MNIST is a dataset of Zalando's article images consisting of 60,000 training examples and 10,000 test examples. Each example comprises a 28×28 grayscale image and an associated label from one of 10 classes.

We load the [FashionMNIST Dataset](#) with the following parameters:

- **root** is the path where the train/test data is stored,
- **train** specifies training or test dataset,

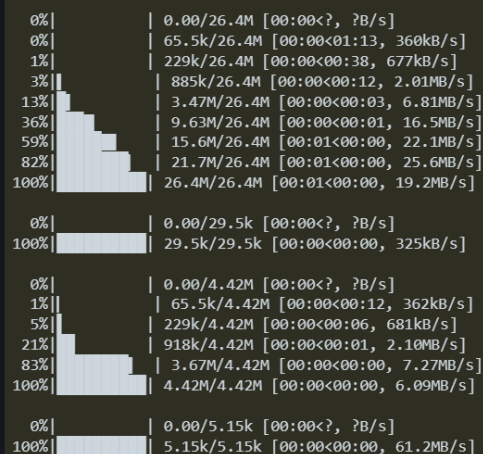
- **download=True** downloads the data from the internet if it's not available at root.
- **transform and target_transform** specify the feature and label transformations

```
import torch
from torch.utils.data import Dataset
from torchvision import datasets
from torchvision.transforms import ToTensor
import matplotlib.pyplot as plt

training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor()
)
```

Out:



The output shows three progress bars for data downloading. Each bar has a percentage on the left, a progress indicator in the middle, and a status string on the right. The status string includes the current progress (e.g., 0.00/26.4M), the total size (26.4M), and the download speed (e.g., 360kB/s).

Progress (%)	Current / Total	Speed
0%	0.00 / 26.4M	00:00<?, ?B/s
0%	65.5k / 26.4M	00:00<01:13, 360kB/s
1%	229k / 26.4M	00:00<00:38, 677kB/s
3%	885k / 26.4M	00:00<00:12, 2.01MB/s
13%	3.47M / 26.4M	00:00<00:03, 6.81MB/s
36%	9.63M / 26.4M	00:00<00:01, 16.5MB/s
59%	15.6M / 26.4M	00:01<00:00, 22.1MB/s
82%	21.7M / 26.4M	00:01<00:00, 25.6MB/s
100%	26.4M / 26.4M	00:01<00:00, 19.2MB/s
0%	0.00 / 29.5k	00:00<?, ?B/s
100%	29.5k / 29.5k	00:00<00:00, 325kB/s
0%	0.00 / 4.42M	00:00<?, ?B/s
1%	65.5k / 4.42M	00:00<00:12, 362kB/s
5%	229k / 4.42M	00:00<00:06, 681kB/s
21%	918k / 4.42M	00:00<00:01, 2.10MB/s
83%	3.67M / 4.42M	00:00<00:00, 7.27MB/s
100%	4.42M / 4.42M	00:00<00:00, 6.09MB/s
0%	0.00 / 5.15k	00:00<?, ?B/s
100%	5.15k / 5.15k	00:00<00:00, 61.2MB/s

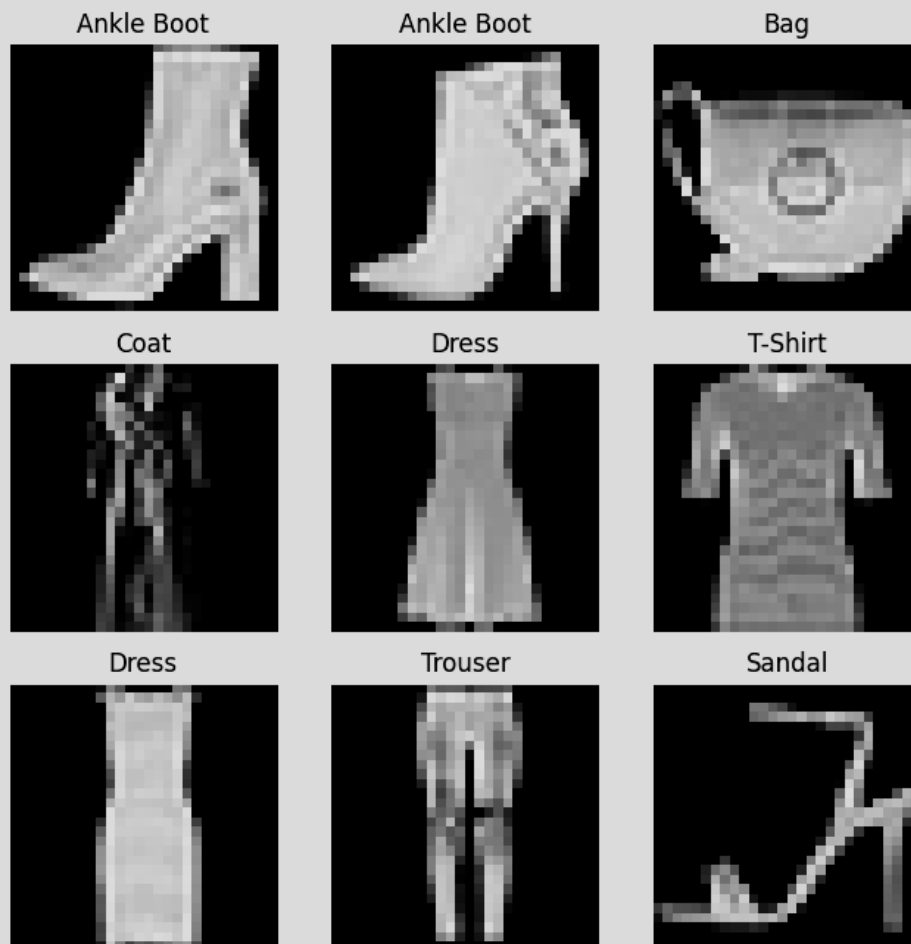
Iterating & Visualising the Dataset:

We can index Datasets manually like a list: **training_data[index]**. We use **matplotlib** to visualize some samples in our training data.

```

labels_map = {
    0: "T-Shirt",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle Boot",
}
figure = plt.figure(figsize=(8, 8))
cols, rows = 3, 3
for i in range(1, cols * rows + 1):
    sample_idx = torch.randint(len(training_data), size=(1,)).item()
    img, label = training_data[sample_idx]
    figure.add_subplot(rows, cols, i)
    plt.title(labels_map[label])
    plt.axis("off")
    plt.imshow(img.squeeze(), cmap="gray")
plt.show()

```



Preparing your data for training with DataLoaders:

The Dataset retrieves our dataset's features and labels one sample at a time. While training a model, we typically want to pass samples in “**minibatches**”, reshuffle the data at every epoch to reduce model overfitting, and use Python's multiprocessing to speed up data retrieval.

DataLoader is an **iterable** that abstracts this complexity for us in an easy API.

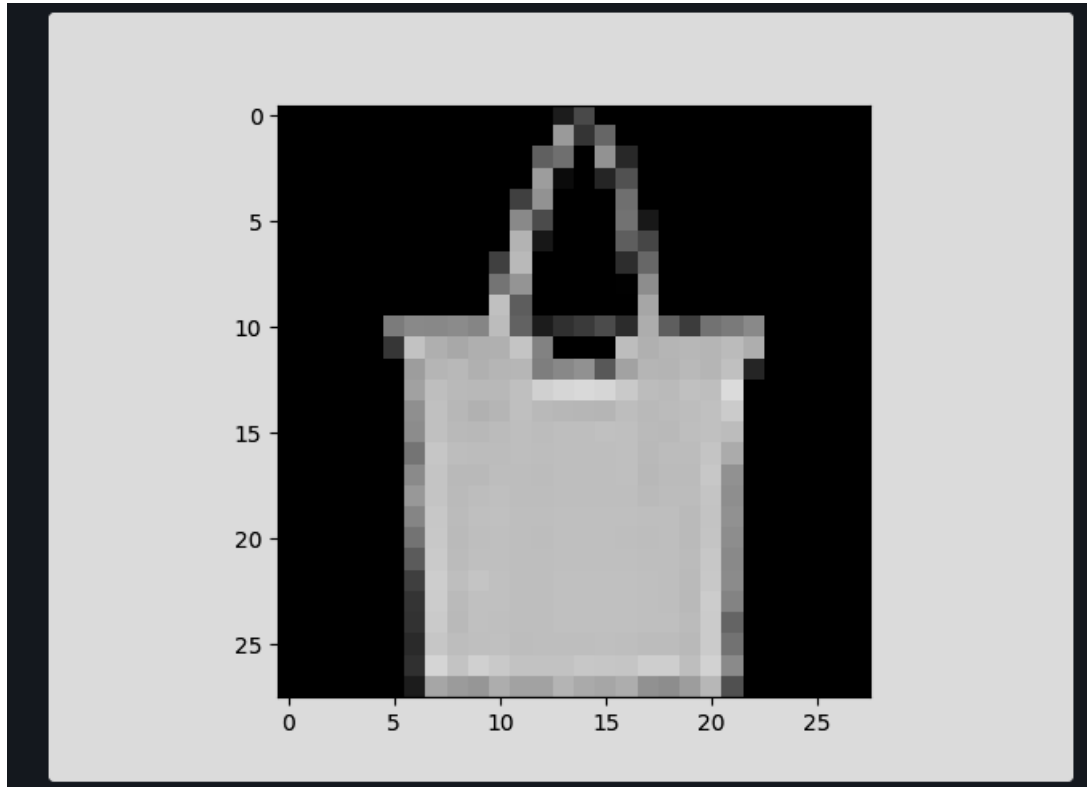
```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=64, shuffle=True)
```

Iterate through the DataLoader:

We have loaded that dataset into the DataLoader and can iterate through the dataset as needed. Each iteration below returns a batch of train_features and train_labels (containing batch_size=64 features and labels respectively). Because we specified shuffle=True, after we iterate over all batches the data is shuffled.

```
# Display image and label.
train_features, train_labels = next(iter(train_dataloader))
print(f"Feature batch shape: {train_features.size()}")
print(f"Labels batch shape: {train_labels.size()}")
img = train_features[0].squeeze()
label = train_labels[0]
plt.imshow(img, cmap="gray")
plt.show()
print(f"Label: {label}")
```



Out:

```
Feature batch shape: torch.Size([64, 1, 28, 28])
Labels batch shape: torch.Size([64])
Label: 8
```

Transforms

Data does not always come in its final processed form that is required for training machine learning algorithms. We use **transforms** to perform some manipulation of the data and make it suitable for training.

All TorchVision datasets have two parameters -transform to modify the features and target_transform to modify the labels - that accept callables containing the transformation logic.

The FashionMNIST features are in PIL Image format, and the labels are integers. For training, we need the features as normalized tensors, and the labels as one-hot encoded tensors. To make these transformations, we use ToTensor and Lambda.

ToTensor():

[ToTensor](#) converts a PIL image or NumPy ndarray into a FloatTensor. and scales the image's pixel intensity values in the range [0., 1.]

Lambda Transforms:

Lambda transforms apply any user-defined lambda function. Here, we define a function to turn the integer into a one-hot encoded tensor. It first creates a zero tensor of size 10 (the number of labels in our dataset) and calls [scatter](#) which assigns a value=1 on the index as given by the label y.

```
import torch
from torchvision import datasets
from torchvision.transforms import ToTensor, Lambda

ds = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor(),
    target_transform=Lambda(lambda y: torch.zeros(10, dtype=torch.float).scatter_(0, torch.tensor(y), value=1))
)
```

Out:

```
0%|          | 0.00/26.4M [00:00<?, ?B/s]
0%|          | 65.5k/26.4M [00:00<01:12, 364kB/s]
1%|          | 229k/26.4M [00:00<00:38, 683kB/s]
3%|          | 786k/26.4M [00:00<00:14, 1.78MB/s]
12%|         | 3.15M/26.4M [00:00<00:03, 6.23MB/s]
34%|         | 8.98M/26.4M [00:00<00:01, 15.6MB/s]
57%|         | 15.0M/26.4M [00:01<00:00, 21.7MB/s]
80%|         | 21.1M/26.4M [00:01<00:00, 29.9MB/s]
93%|         | 24.5M/26.4M [00:01<00:00, 26.5MB/s]
100%|        | 26.4M/26.4M [00:01<00:00, 19.4MB/s]

0%|          | 0.00/29.5k [00:00<?, ?B/s]
100%|         | 29.5k/29.5k [00:00<00:00, 326kB/s]

0%|          | 0.00/4.42M [00:00<?, ?B/s]
1%|          | 65.5k/4.42M [00:00<00:12, 361kB/s]
5%|          | 229k/4.42M [00:00<00:06, 679kB/s]
20%|         | 885k/4.42M [00:00<00:01, 2.02MB/s]
79%|         | 3.47M/4.42M [00:00<00:00, 6.85MB/s]
100%|        | 4.42M/4.42M [00:00<00:00, 6.07MB/s]

0%|          | 0.00/5.15k [00:00<?, ?B/s]
100%|         | 5.15k/5.15k [00:00<00:00, 60.0MB/s]
```

Lab Tasks

Tensors:

1. Create a tensor of numbers from 1 to 12. Reshape it into 3x4 and 4x3. Print original and new shapes. Finally flatten the tensor back to 1D.
2. Create two random 3x3 tensors. Compare both tensors and find positions where values of first tensor are greater than the value of second tensor. Count how many such positions exist. Print the Boolean tensor result.
3. Create two tensors of size 2x3. Concatenate them once along rows (dim = 0) and once along columns (dim = 1). Print resulting shapes.

Dataset & DataLoader:

4. Load the MNIST dataset using PyTorch's Dataset. Print the length of training & test dataset.
5. Access anyone sample from the dataset using indexing. Print image shape and label. Display the image using matplotlib.
6. Create a DataLoader for training dataset with batch_size = 32 and shuffle = True.
7. Iterate through first three batches. Print batch number and label values of each batch.

Transforms:

8. Use transforms.Compose() to combine two transforms: ToTensor() and Normalize(). Load dataset with combined transforms. Print shape of one sample.